

Sistem pentru evidenta cartilor dintr-o biblioteca

1. Descriere generala

Aplicatia reprezinta un sistem de gestiune a cartilor dintr-o biblioteca si permite administrarea unei colectii de carti utilizand exclusiv argumente din linia de comanda.

Scopul proiectului este evidenta cartilor disponibile, cautarea acestora dupa diferite criterii, precum si gestionarea procesului de imprumut si returnare.

Datele sunt pastrate permanent intr-un fisier text (`books.txt`), astfel incat informatiile sa fie disponibile intre mai multe rulari ale aplicatiei.

Functionalitati implementate:

- adaugarea unei carti noi;
- afisarea tuturor cartilor;
- cautarea dupa titlu;
- cautarea dupa autor;
- afisarea cartilor disponibile;
- afisarea cartilor imprumutate;
- imprumutarea unei carti;
- returnarea unei carti;
- validarea argumentelor din linia de comanda;
- persistenta datelor in fisier.

2. Membrii echipei si responsabilitati

Stefan Deaca

Responsabilitati:

- analiza cerintelor proiectului;
- proiectarea arhitecturii aplicatiei;
- implementarea clasei `Library`;
- implementarea functiilor de cautare;
- implementarea functiilor de afisare;
- implementarea interfetei CLI (`CliApplication`);
- testarea aplicatiei;
- realizarea documentatiei tehnice.

Darius Rozemberg

Responsabilitati:

- implementarea clasei `Book`;
- implementarea clasei `FileManager`;

- implementarea persistentei datelor in fisiere;
 - implementarea operatiilor de imprumut si returnare la nivelul obiectelor Book;
 - implementarea mecanismelor de citire si scriere in fisier;
 - testarea si depanarea aplicatiei.
-

3. Structura claselor

Clasa Book

Reprezinta o carte din biblioteca.

Attribute

- `id`
- `title`
- `author`
- `type`
- `location`
- `isBorrowed`
- `borrowTime`

Metode

- `borrow()`
 - `returnBook()`
 - `isAvailable()`
 - `getDetails()`
 - `isValidField()`
 - `operator<<`
-

Clasa Library

Gestioneaza colectia de carti si logica principala a aplicatiei.

Attribute

- `vector<unique_ptr<Book>> books`
- `FileManager fileManager`

Metode

- `loadBooks()`
- `saveBooks()`
- `addBook()`
- `searchByTitle()`
- `searchByAuthor()`
- `displayBooks()`

- `displayAvailable()`
 - `displayBorrowed()`
 - `borrowBook()`
 - `returnBook()`
 - `findById()`
 - `nextId()`
-

Clasa FileManager

Responsabila pentru persistenta datelor.

Metode

- `readBooksFromFile()`
 - `writeBooksToFile()`
 - `updateBook()`
-

Clasa CliApplication

Gestioneaza interactiunea cu utilizatorul prin argumente in linia de comanda.

Metode

- `run()`
 - `dispatch()`
 - `printUsage()`
 - `parsePositiveId()`
 - `validateAddBookArgs()`
-

4. Relatiile dintre clase

Compozitie

Library contine o colectie de obiecte Book.

```
Library
|
+--- Book
+--- Book
+--- Book
```

Daca obiectul Library este distrus, colectia de carti gestionata de acesta nu mai este utilizata.

Asociere

Library utilizeaza FileManager pentru operatiile de persistenta.

```
Library ----> FileManager
```

Asociere

CliApplication utilizeaza Library pentru executarea comenzilor primite.

```
CliApplication ----> Library
```

5. Structura fisierului books.txt

Fisierul este utilizat pentru stocarea permanenta a informatiilor despre carti.

Format:

```
id|title|author|type|location|isBorrowed|borrowTime
```

Exemplu:

```
1|The C Programming Language|Kernighan & Ritchie|Technical|A-  
12|0|0000000000  
2|Clean Code|Robert C. Martin|Software|B-04|1|1776456871
```

Descriere campuri:

- **id** - identificator unic;
- **title** - titlul cartii;
- **author** - autorul;
- **type** - categoria cartii;
- **location** - locatia fizica;
- **isBorrowed** - 0 disponibil, 1 imprumutat;
- **borrowTime** - momentul imprumutului.

6. Structura proiectului

```
include/  
├── Book.h  
├── Library.h  
├── FileManager.h  
└── CliApplication.h
```

```
src/
├── Book.cpp
├── Library.cpp
├── FileManager.cpp
├── CliApplication.cpp
├── main.cpp
└──

books.txt
README.md
docs.md
Makefile
```

7. Comenzi disponibile

Help

```
./build/library -help
./build/library --help
./build/library help
```

Afiseaza toate comenzile disponibile.

Afisare toate cartile

```
./build/library list
```

Adaugare carte

```
./build/library add-book "<title>" "<author>" "<type>" "<location>"
```

Exemplu:

```
./build/library add-book "Clean Code" "Robert Martin" "Software" "B-04"
```

Cautare dupa titlu

```
./build/library search-title "clean"
```

Cautare dupa autor

```
./build/library search-author "martin"
```

Afisare carti disponibile

```
./build/library search-available
```

Afisare carti imprumutate

```
./build/library search-borrowed
```

Imprumutare carte

```
./build/library borrow 3
```

Returnare carte

```
./build/library return 3
```

8. Validari implementate

Aplicatia verifica:

- existenta argumentelor obligatorii;
- ID pozitiv pentru comenzile borrow si return;
- campuri necomplete;
- prezenta caracterului | in datele introduse;
- existenta cartii cautate;
- disponibilitatea cartii la imprumut;

- starea de imprumut la returnare.
-

9. Elemente tehnice utilizate

Pentru implementare au fost folosite:

- Programare Orientata pe Obiecte (OOP);
 - STL (`vector`, `string`);
 - Smart pointers (`unique_ptr`);
 - Operator supraincarcat (`operator<<`);
 - Functie template (`displayMatching`);
 - Citire si scriere in fisiere;
 - Parsarea argumentelor din linia de comanda.
-

10. Concluzii

Proiectul implementeaza un sistem complet pentru evidenta cartilor dintr-o biblioteca utilizand concepte moderne C++.

Aplicatia respecta cerintele proiectului, foloseste clase, STL, fisiere pentru persistenta datelor, validari pentru argumente si relatii intre clase, fiind usor de extins cu functionalitati suplimentare in viitor.